

UNIVERSIDAD AMERICANA
FACULTAD DE INGENIERÍA Y ARQUITECTURA



Ingeniería en Sistemas de Información

SIS0421 - Servicios Web

Período: VIII Semestre

Práctica guiada 1 y 2 - Persistencia y relaciones con Spring Data JPA

Estudiantes:

Diego Andrés Martínez Castillo

Docente: MSc. José Durán García

Managua, 06 de octubre de 2026

Tabla de Contenido

Práctica Guiada 1.....	1
1. Configuración de la conexión.....	1
2. Estructura de paquetes.....	1
3. Entidades desarrolladas.....	1
4. Diagrama de relaciones.....	1
5. Scripts V1, V2 y V3.....	1
6. Capturas de Postman.....	1
7. Evidencia de PostgreSQL.....	1
8. Respuestas de comprobación.....	1
8.1. ¿Cuál es la función de Spring Data JPA?.....	1
8.2. ¿Qué función cumple Hibernate?.....	1
8.3. ¿Qué diferencia existe entre JPA y Hibernate?.....	1
8.4. ¿Qué función cumple `@Entity`?.....	1
8.5. ¿Qué función cumple `@ManyToOne`?.....	1
8.6. ¿Qué función cumple `@JoinColumn`?.....	1
8.7. ¿Qué función cumple `JpaRepository`?.....	1
8.8. ¿Por qué se utilizan migraciones?.....	1
8.9. ¿Qué diferencia existe entre V1, V2 y V3?.....	1
8.10. ¿Qué problema puede producir una relación bidireccional al generar JSON?.....	1
9. Conclusión.....	1
Práctica Guiada 2.....	2
10. Estructura Controller, Service, Repository, Entity y DTO.....	2
11. ProductoService.....	2
12. ProductoRequestDTO.....	2
13. CRUD completo de productos.....	2
14. Evidencias GET, POST, PUT y DELETE en Postman.....	2
15. Consulta de productos por categoría.....	2
16. Migración `V4__crear_etiquetas.sql`.....	2
17. Entidad y repositorio Etiqueta.....	2
18. Relación Producto N – N Etiqueta.....	2
19. Tabla `producto_etiqueta` en PostgreSQL.....	2
20. Asociación de etiquetas mediante Postman.....	2
21. Solución del reto final.....	2
22. Respuestas de comprobación.....	2

22.1. ¿Cuál es la función de una clase Service?.....	2
22.2. ¿Por qué un controlador no debería contener toda la lógica de negocio?.....	2
22.3. ¿Qué es un DTO?.....	2
22.4. ¿Qué diferencia existe entre un DTO y una entidad JPA?.....	2
22.5. ¿Qué ventaja tiene recibir `categoriald` en lugar de una entidad Categoria completa?.....	2
22.6. ¿Qué relación representan `@OneToMany` y `@ManyToOne`?.....	2
22.7. ¿Dónde se almacena la clave foránea en la relación Categoria-Producto?.....	2
22.8. ¿Qué representa `@ManyToMany`?.....	2
22.9. ¿Cuál es la función de `@JoinTable`?.....	2
22.10. ¿Por qué se necesita la tabla `producto_etiqueta`?.....	2
22.11. ¿Qué responsabilidad corresponde al Repository?.....	2
22.12. Explique el flujo: Cliente → Controller → Service → Repository → PostgreSQL.....	2

Práctica Guiada 1

1. Configuración de la conexión

El proyecto se configuró para conectarse a PostgreSQL mediante el archivo `src/main/resources/application.properties`.

```
spring.application.name=gestion-productos

spring.datasource.url=jdbc:postgresql://localhost:5432/gestion_productos
spring.datasource.username=postgres
spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

spring.flyway.enabled=true
```

La base de datos utilizada se llama `gestion_productos` y se ejecuta en PostgreSQL mediante el puerto predeterminado 5432.

La propiedad `spring.jpa.hibernate.ddl-auto=validate` permite validar que las entidades Java coincidan con la estructura creada por las migraciones. Flyway está habilitado para aplicar las migraciones SQL de manera ordenada.

2. Estructura de paquetes

El proyecto se organizó usando una estructura por responsabilidades:

```
ni.edu.uam.gestion_productos
|
|— controllers
|   |— CategoriaController.java
|   |— ProductoController.java
|   |— ProveedorController.java
|   |— EtiquetaController.java
|
|— dto
|   |— ProductoRequestDTO.java
|
|— models
|   |— Categoria.java
|   |— Producto.java
|   |— Proveedor.java
|   |— Etiqueta.java
|
|— repositories
|   |— CategoriaRepository.java
|   |— ProductoRepository.java
|   |— ProveedorRepository.java
|   |— EtiquetaRepository.java
|
|— services
|   |— ProductoService.java
|
|— GestionProductosApplication.java
```

Los controladores reciben las solicitudes HTTP. Los servicios contienen lógica de negocio. Los repositorios permiten acceder a PostgreSQL mediante Spring Data JPA. Las entidades representan las tablas de la base de datos. El DTO se utiliza para recibir datos específicos al crear o actualizar productos.

3. Entidades desarrolladas

Categoria

La entidad Categoria representa la tabla categoria. Contiene los campos id, nombre y activa.

Una categoría puede tener varios productos mediante la relación:

```
@OneToMany(mappedBy = "categoria")
@JsonIgnore
private List<Producto> productos = new ArrayList<>();
```

Producto

La entidad Producto contiene los campos id, codigo, nombre, precioVenta y existencia.

Cada producto pertenece a una categoría:

```
@ManyToOne(optional = false)
@JoinColumn(name = "categoria_id", nullable = false)
private Categoria categoria;
```

Un producto puede pertenecer a un proveedor:

```
@ManyToOne
@JoinColumn(name = "proveedor_id")
private Proveedor proveedor;
```

Además, un producto puede tener varias etiquetas:

```
@ManyToMany
@JoinTable(
    name = "producto_etiqueta",
    joinColumns = @JoinColumn(name = "producto_id"),
    inverseJoinColumns = @JoinColumn(name = "etiqueta_id")
)
private Set<Etiqueta> etiquetas = new HashSet<>();
```

Proveedor

La entidad Proveedor contiene id, nombre, telefono, correo y activo.

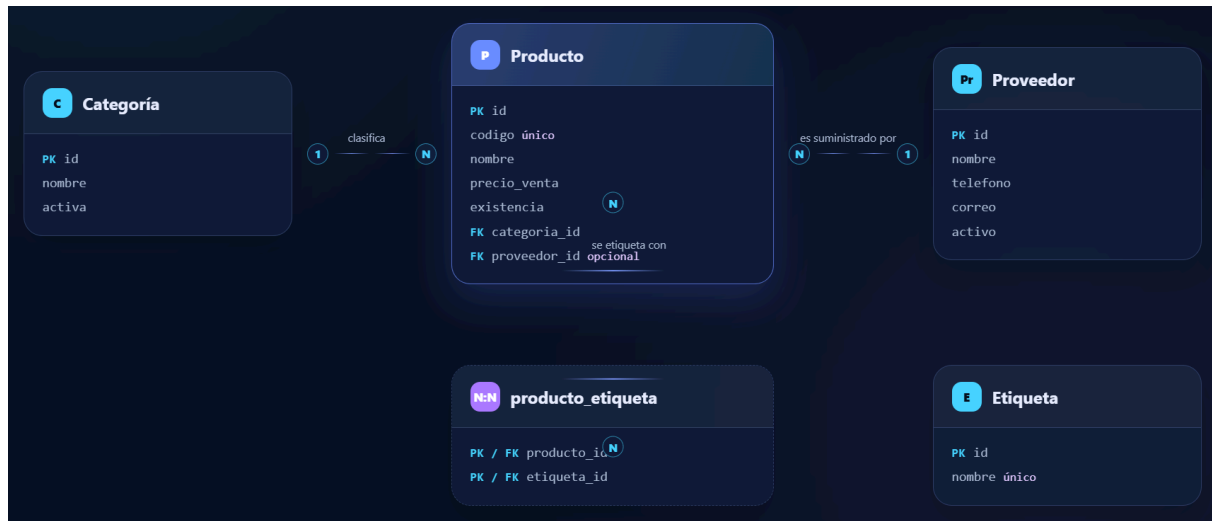
Un proveedor puede estar relacionado con varios productos:

```
@OneToMany(mappedBy = "proveedor", cascade = CascadeType.PERSIST)
@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
private List<Producto> productos = new ArrayList<>();
```

Etiqueta

La entidad Etiqueta contiene id y nombre. Su nombre es único en la base de datos.

4. Diagrama de relaciones



5. Scripts V1, V2 y V3

V1__crear_tablas.sql

```
CREATE TABLE categoria (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  activa BOOLEAN NOT NULL DEFAULT TRUE  
);
```

```
CREATE TABLE producto (  
  id SERIAL PRIMARY KEY,  
  codigo VARCHAR(30) NOT NULL UNIQUE,  
  nombre VARCHAR(150) NOT NULL,  
  precio_venta NUMERIC(12,2) NOT NULL,  
  existencia INTEGER NOT NULL DEFAULT 0,  
  categoria_id INTEGER NOT NULL,  
  
  CONSTRAINT fk_producto_categoria  
    FOREIGN KEY (categoria_id)  
    REFERENCES categoria(id)  
);
```

V2__agregar_descripcion_producto.sql

```
ALTER TABLE producto  
ADD COLUMN descripcion VARCHAR(500);
```

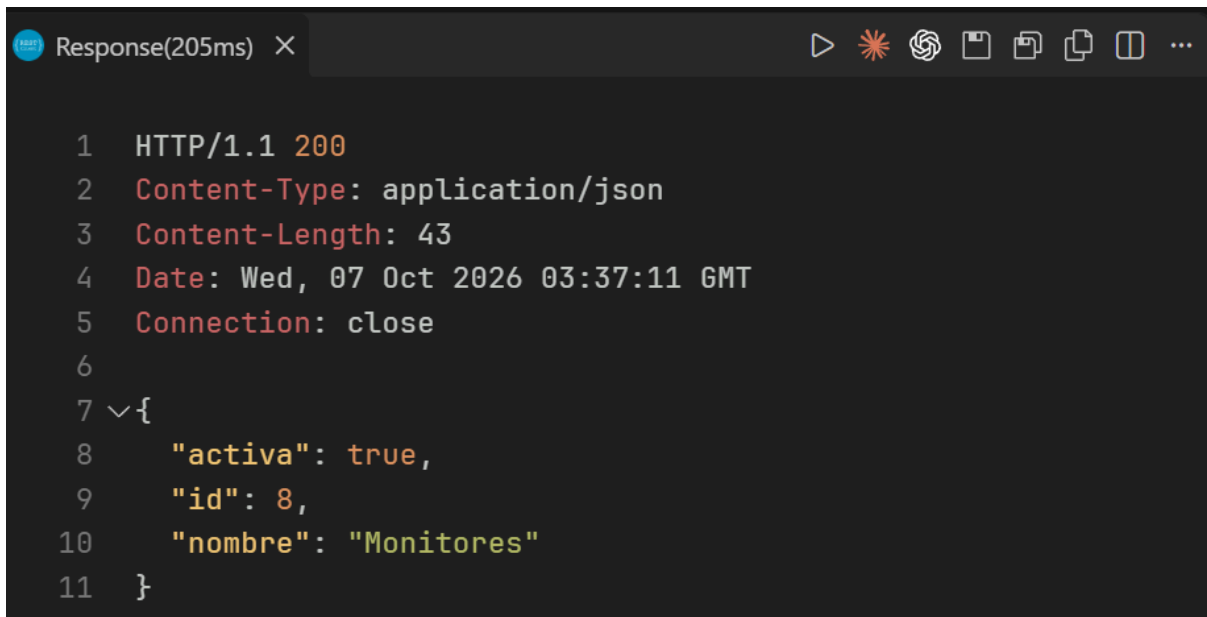
V3__crear_proveedor.sql

```
CREATE TABLE proveedor (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(150) NOT NULL,  
  telefono VARCHAR(30),  
  correo VARCHAR(254),  
  activo BOOLEAN NOT NULL DEFAULT TRUE  
);
```

```
ALTER TABLE producto  
ADD COLUMN proveedor_id INTEGER,  
ADD CONSTRAINT fk_producto_proveedor  
  FOREIGN KEY (proveedor_id) REFERENCES proveedor(id);
```

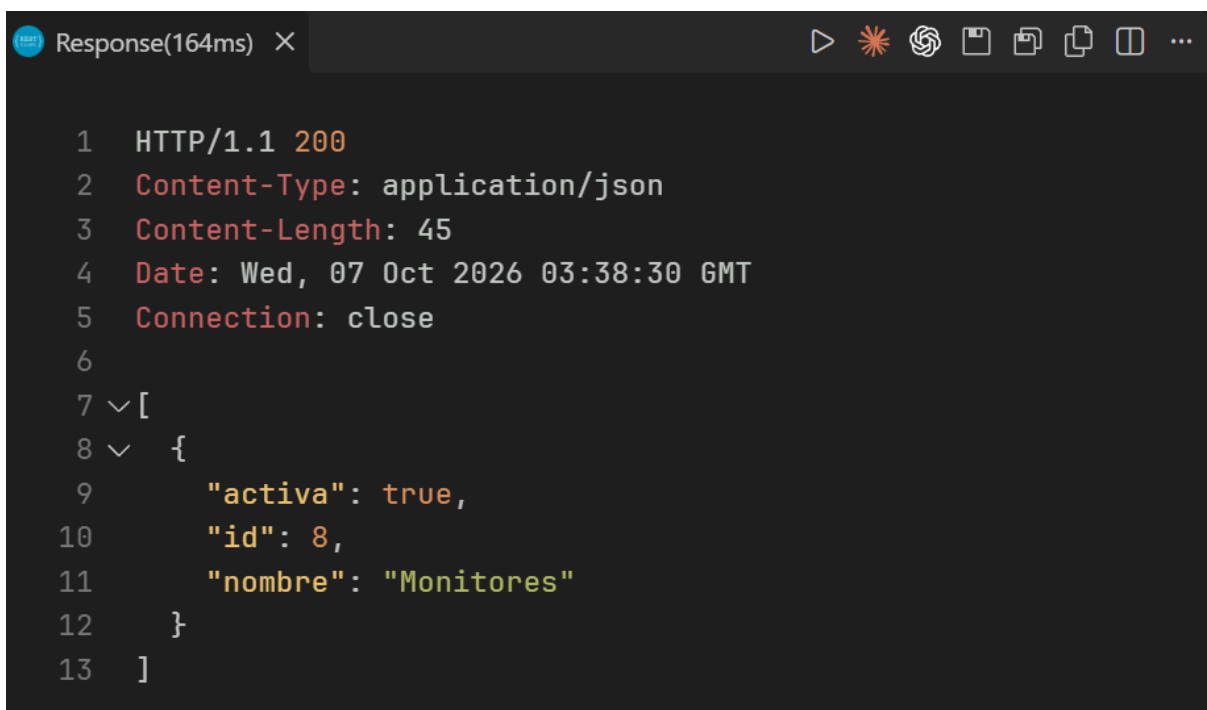
```
CREATE INDEX idx_producto_proveedor_id ON producto(proveedor_id);
```

6. Capturas de Postman



Response(205ms) X

```
1 HTTP/1.1 200
2 Content-Type: application/json
3 Content-Length: 43
4 Date: Wed, 07 Oct 2026 03:37:11 GMT
5 Connection: close
6
7 {
8   "activa": true,
9   "id": 8,
10  "nombre": "Monitores"
11 }
```

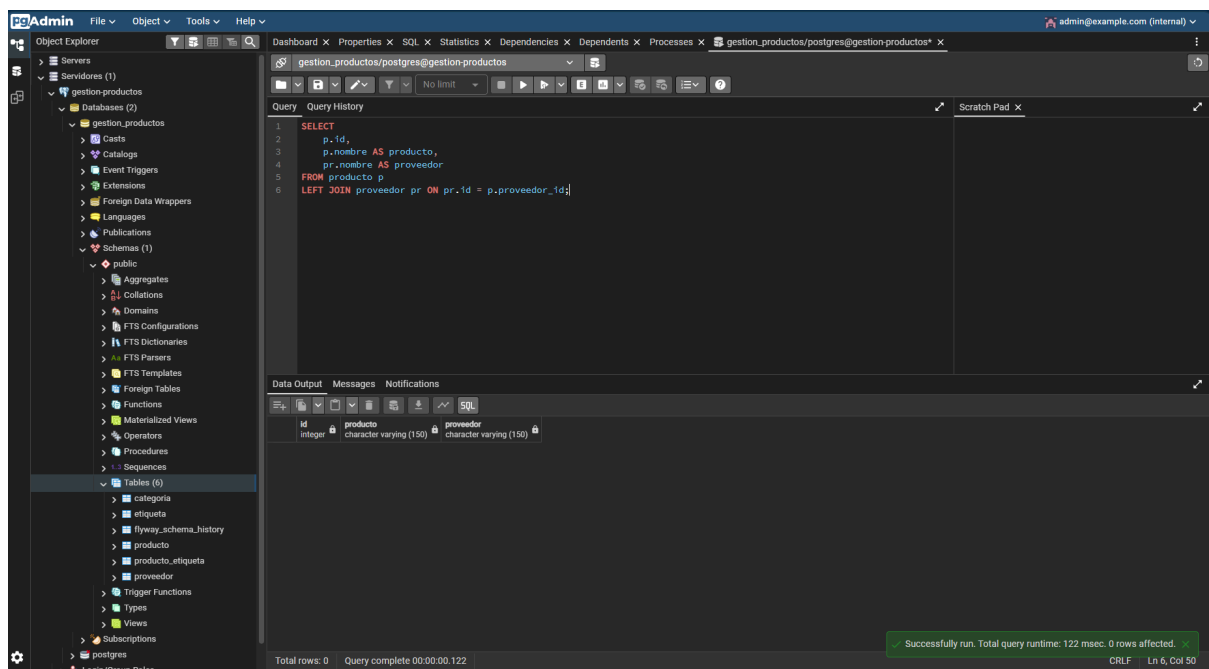
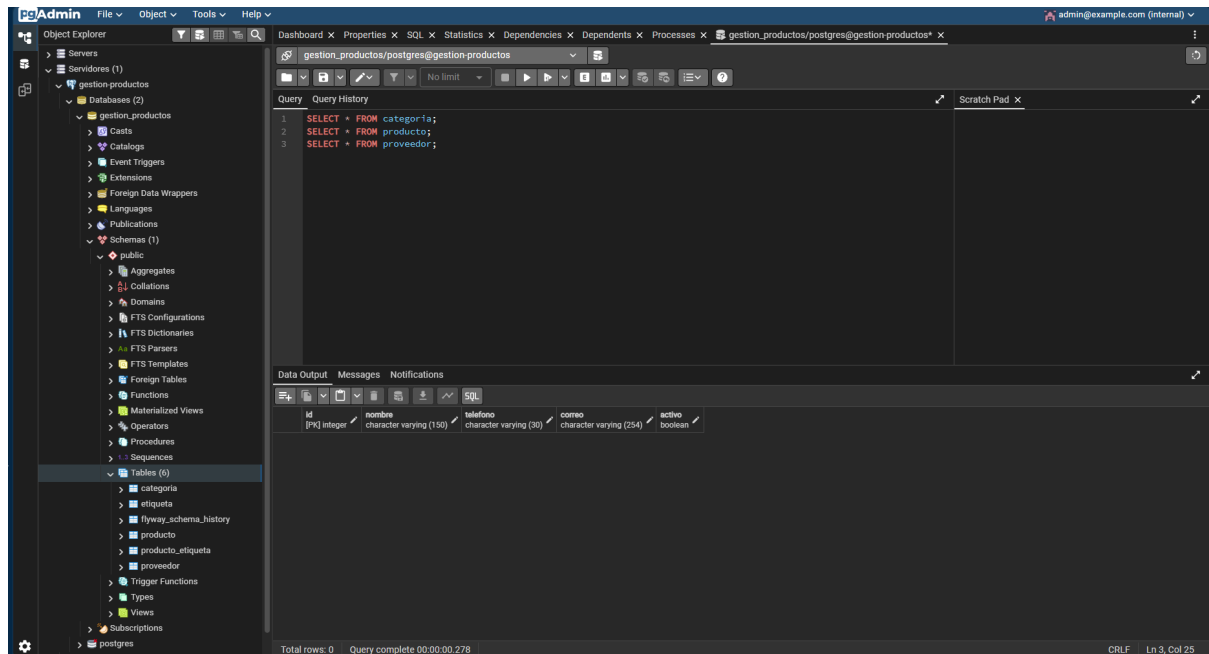


Response(164ms) X

```
1 HTTP/1.1 200
2 Content-Type: application/json
3 Content-Length: 45
4 Date: Wed, 07 Oct 2026 03:38:30 GMT
5 Connection: close
6
7 [
8   {
9     "activa": true,
10    "id": 8,
11    "nombre": "Monitores"
12  }
13 ]
```

7. Evidencia de PostgreSQL

La base de datos contiene las tablas creadas mediante Flyway. Se verificó la existencia de registros de categorías, productos y proveedores.



8. Respuestas de comprobación

8.1. ¿Cuál es la función de Spring Data JPA?

Simplificar el acceso a datos en aplicaciones Spring. Permite crear repositorios que realizan operaciones CRUD y consultas sin escribir manualmente gran parte del código SQL.

8.2. ¿Qué función cumple Hibernate?

Es una implementación de JPA. Se encarga de convertir objetos Java en registros de base de datos y viceversa. También genera consultas SQL y administra las relaciones entre entidades.

8.3. ¿Qué diferencia existe entre JPA y Hibernate?

JPA es una especificación o conjunto de reglas para persistencia en Java. Hibernate es una herramienta que implementa esa especificación. En este proyecto se utilizan las anotaciones JPA y Hibernate realiza el trabajo de persistencia.

8.4. ¿Qué función cumple `@Entity`?

Indica que una clase Java representa una tabla de base de datos y que sus objetos pueden almacenarse mediante JPA.

8.5. ¿Qué función cumple `@ManyToOne`?

Representa una relación muchos a uno. En este proyecto, varios productos pueden pertenecer a una categoría y varios productos pueden pertenecer a un proveedor.

8.6. ¿Qué función cumple `@JoinColumn`?

Indica la columna que almacena la clave foránea. Por ejemplo, `categoria_id` en la tabla `producto` apunta a la tabla `categoria`.

8.7. ¿Qué función cumple `JpaRepository`?

Proporciona métodos ya implementados para guardar, listar, buscar, actualizar y eliminar entidades, por ejemplo `save`, `findAll`, `findById` y `deleteById`.

8.8. ¿Por qué se utilizan migraciones?

Porque permiten controlar los cambios de la estructura de la base de datos mediante archivos versionados. Esto asegura que todos los desarrolladores y ambientes utilicen el mismo esquema.

8.9. ¿Qué diferencia existe entre V1, V2 y V3?

V1 crea las tablas iniciales de categoría y producto. V2 agrega la columna descripción a producto. V3 crea la tabla proveedor y agrega la relación entre proveedor y producto.

8.10. ¿Qué problema puede producir una relación bidireccional al generar JSON?

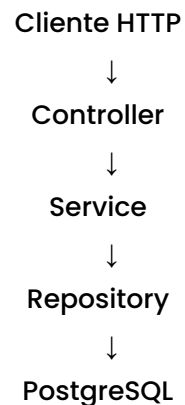
Puede producir una referencia circular. Por ejemplo, un producto contiene una categoría y la categoría contiene una lista de productos; al convertir la respuesta a JSON, el proceso puede repetirse indefinidamente. Para evitarlo se utilizó `@JsonIgnore` o `@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)`.

9. Conclusión

Durante la práctica se configuró una aplicación Spring Boot conectada a PostgreSQL. Se aplicaron migraciones con Flyway, se crearon entidades JPA, repositorios y controladores REST. También se implementaron relaciones entre categorías, productos y proveedores. La práctica permitió comprender cómo Spring Data JPA y Hibernate facilitan la persistencia de datos y cómo Flyway mantiene versionada la estructura de la base de datos.

Práctica Guiada 2

10. Estructura Controller, Service, Repository, Entity y DTO



El Controller recibe solicitudes HTTP. El Service contiene reglas y procesos de negocio. El Repository accede a la base de datos. La Entity representa una tabla. El DTO transporta datos entre el cliente y la aplicación sin exponer directamente toda la entidad.

11. ProductoService

ProductoService concentra la lógica de negocio de productos.

Entre sus métodos se encuentran:

```
public List<Producto> listar()
public Producto buscarPorId(Integer id)
public Producto guardar(ProductoRequestDTO dto)
public Producto actualizar(Integer id, ProductoRequestDTO dto)
public void eliminar(Integer id)
public List<Producto> listarPorCategoria(Integer categoriaId)
public Producto agregarEtiqueta(Integer productoid, Integer etiquetaid)
public void eliminarEtiqueta(Integer productoid, Integer etiquetaid)
public List<Producto> listarPorEtiqueta(Integer etiquetaid)
```

El servicio busca categorías y etiquetas mediante sus repositorios antes de realizar operaciones que requieran esas relaciones.

12. ProductoRequestDTO

ProductoRequestDTO se utiliza para recibir los datos necesarios para crear o actualizar un producto.

```
public class ProductoRequestDTO {  
  
    private String codigo;  
    private String nombre;  
    private BigDecimal precioVenta;  
    private Integer existencia;  
    private Integer categoriald;  
  
    // Getters y setters  
}
```

El DTO recibe únicamente el identificador de la categoría, en vez de una entidad Categoría completa.

13. CRUD completo de productos

Operación	Método	Endpoint
Listar productos	GET	/api/productos
Buscar un producto	GET	/api/productos/{id}
Crear producto	POST	/api/productos
Actualizar producto	PUT	/api/productos/{id}
Eliminar producto	DELETE	/api/productos/{id}

14. Evidencias GET, POST, PUT y DELETE en Postman

```
Response(20ms) X
1 HTTP/1.1 200
2 Content-Type: application/json
3 Content-Length: 2
4 Date: Wed, 07 Oct 2026 03:44:37 GMT
5 Connection: close
6
7 []
```

15. Consulta de productos por categoría

Se implementó el método en el repositorio:

```
List<Producto> findByCategoriald(Integer categoriald);
```

En el servicio

```
public List<Producto> listarPorCategoría(Integer categoriald) {
    return productoRepository.findByCategoriald(categoriald);
}
```

Endpoint

```
GET /api/productos/categoría/{categoriald}
```

16. Migración V4__crear_etiquetas.sql

La migración V4 crea la tabla etiqueta y la tabla intermedia producto_etiqueta.

```
CREATE TABLE etiqueta (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE producto_etiqueta (
```

```

producto_id INTEGER NOT NULL,
etiqueta_id INTEGER NOT NULL,

PRIMARY KEY (producto_id, etiqueta_id),

CONSTRAINT fk_producto_etiqueta_producto
FOREIGN KEY (producto_id)
REFERENCES producto(id),

CONSTRAINT fk_producto_etiqueta_etiqueta
FOREIGN KEY (etiqueta_id)
REFERENCES etiqueta(id)
);

```

17. Entidad y repositorio Etiqueta

La entidad Etiqueta representa la tabla etiqueta.

```

@Entity
@Table(name = "etiqueta")
public class Etiqueta {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false, unique = true, length = 100)
    private String nombre;
}

```

El repositorio permite realizar operaciones CRUD:

```

public interface EtiquetaRepository extends JpaRepository<Etiqueta, Integer> {
}

```

El Endpoint para crear la etiqueta es:

```
POST /api/etiquetas
```

18. Relación Producto N – N Etiqueta

La relación muchos a muchos se definió en Producto.java.

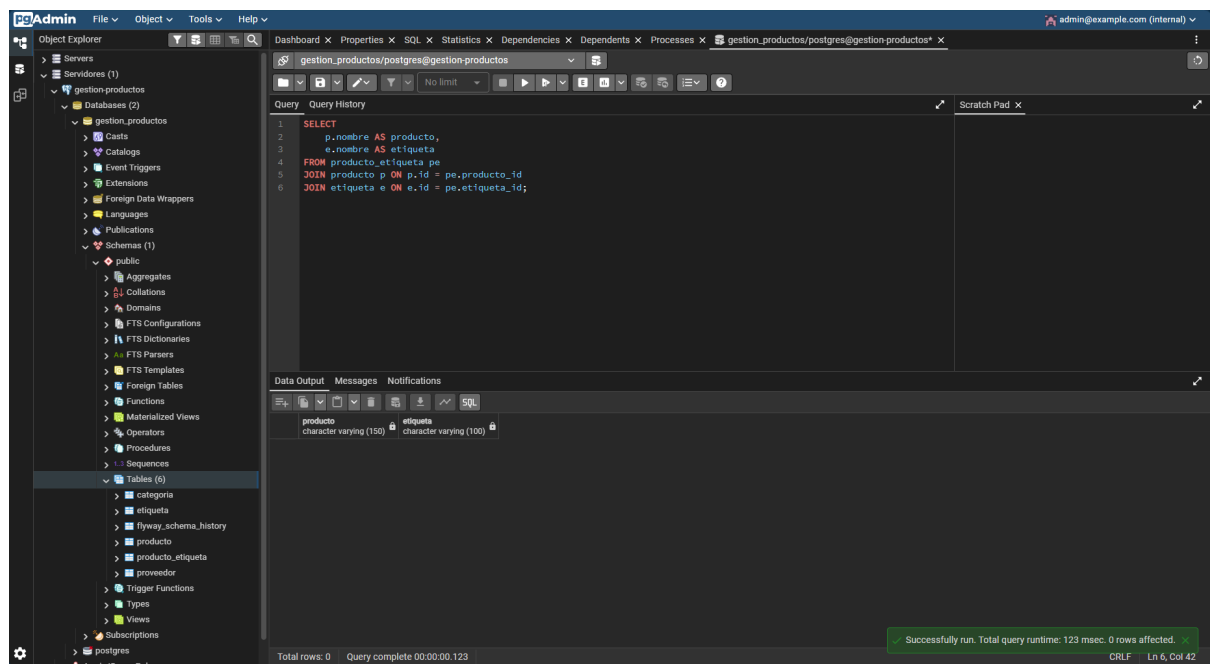
```
@ManyToMany
@JoinTable(
    name = "producto_etiqueta",
    joinColumns = @JoinColumn(name = "producto_id"),
    inverseJoinColumns = @JoinColumn(name = "etiqueta_id")
)
private Set<Etiqueta> etiquetas = new HashSet<>();
```

19. Tabla `producto\_etiqueta` en PostgreSQL

La tabla producto_etiqueta almacena las asociaciones entre productos y etiquetas.

Columna	Descripcion
producto_id	Identificador del producto
etiqueta_id	Identificador de la etiqueta

La clave primaria está compuesta por ambas columnas. De esta manera no se puede repetir la misma asociación entre un producto y una etiqueta.



20. Asociación de etiquetas mediante Postman

Para agregar una etiqueta a un producto se implementó:

POST /api/productos/{productoid}/etiquetas/{etiquetaid}

Ejemplo:

POST /api/productos/2/etiquetas/1

GET /api/productos/etiqueta/{etiquetaid}

GET /api/productos/etiqueta/1

DELETE /api/productos/{productoid}/etiquetas/{etiquetaid}

DELETE /api/productos/2/etiquetas/1

Este DELETE elimina solamente el registro de la tabla producto_etiqueta. No elimina el producto ni la etiqueta.

21. Solución del reto final

El reto final consistió en implementar una relación muchos a muchos entre productos y etiquetas.

La solución incluyó:

Crear la entidad Etiqueta.

Crear EtiquetaRepository.

Crear la migración V4__crear_etiquetas.sql.

Crear la tabla intermedia producto_etiqueta.

Agregar @ManyToMany y @JoinTable en la entidad Producto.

Crear el endpoint POST /api/etiquetas.

Crear el método para asociar etiquetas a productos.

Crear el endpoint para eliminar únicamente la asociación.

Crear el endpoint para consultar productos por etiqueta.

Con esta solución, el sistema permite clasificar productos usando varias etiquetas y consultar los productos asociados a cada una.

22. Respuestas de comprobación

22.1. ¿Cuál es la función de una clase Service?

La clase Service contiene la lógica de negocio. Coordina repositorios, valida información y define procesos antes de guardar, actualizar o eliminar datos.

22.2. ¿Por qué un controlador no debería contener toda la lógica de negocio?

El controlador debe encargarse de recibir solicitudes HTTP y devolver respuestas. Si contiene toda la lógica, el código se vuelve difícil de mantener, probar y reutilizar. Separar la lógica en un Service mejora la organización.

22.3. ¿Qué es un DTO?

DTO significa Data Transfer Object. Es una clase utilizada para transportar datos entre el cliente y la aplicación.

22.4. ¿Qué diferencia existe entre un DTO y una entidad JPA?

Una entidad JPA representa una tabla de base de datos y define cómo se persiste la información. Un DTO representa los datos que se reciben o envían por la API. El DTO no necesita contener todas las relaciones o campos de la entidad.

22.5. ¿Qué ventaja tiene recibir ``categoriaId`` en lugar de una entidad `Categoria` completa?

Se envía menos información desde el cliente. El servicio busca la categoría real en la base de datos usando el ID, valida que exista y evita que el cliente modifique campos de la categoría accidentalmente.

22.6. ¿Qué relación representan ``@OneToMany`` y ``@ManyToOne``?

Representan una relación uno a muchos y muchos a uno. Una categoría tiene varios productos y cada producto pertenece a una categoría.

22.7. ¿Dónde se almacena la clave foránea en la relación `Categoria-Producto`?

La clave foránea se almacena en la tabla `producto`, en la columna `categoria_id`.

22.8. ¿Qué representa ``@ManyToMany``?

Representa una relación muchos a muchos. En este proyecto, un producto puede tener muchas etiquetas y una etiqueta puede pertenecer a muchos productos.

22.9. ¿Cuál es la función de ``@JoinTable``?

`@JoinTable` define la tabla intermedia utilizada para guardar una relación muchos a muchos. En este caso define la tabla `producto_etiqueta` y sus columnas `producto_id` y `etiqueta_id`.

22.10. ¿Por qué se necesita la tabla ``producto_etiqueta``?

Porque una relación muchos a muchos no puede almacenarse con una sola clave foránea en una de las dos tablas. La tabla intermedia registra cada asociación entre un producto y una etiqueta.

22.11. ¿Qué responsabilidad corresponde al `Repository`?

El `Repository` se encarga del acceso a datos. Permite guardar, buscar, listar y eliminar entidades en la base de datos.

22.12. Explique el flujo: Cliente → Controller → Service → Repository → PostgreSQL

El cliente realiza una solicitud HTTP. El Controller recibe la solicitud y extrae los datos. El Service aplica la lógica de negocio y valida la información. El Repository ejecuta las operaciones de persistencia. Finalmente, PostgreSQL almacena o consulta los datos y la respuesta regresa al cliente.ar